

AI Product Intelligence System

Gen AI Bootcamp — Day 2 Homework Submission

Advanced Features: Complementary Recommendations, Duplicate Catalog Cleanup & Text-Based Reverse Search

Author	Manish
Team	Lambda Lords
Dataset	Fashion Product Images (Small) — Kaggle
Model Used	CLIP (openai/clip-vit-base-patch32)
Kaggle Notebook	kaggle.com/code/manishr00799/notebook4d97fc13a9

Overview

The base system understands products from images, generates product metadata, and finds visually similar products using CLIP embeddings and vector search. This submission extends that system with all three advanced features from the Day 2 challenge: a complementary-product recommendation engine, an automatic duplicate/near-duplicate catalog cleaner, and a text-to-image reverse search interface.

Approach in Brief

- Loaded 200 sampled products from styles.csv and matched each to its image on disk, skipping any missing files.
- Generated a 512-dimensional, L2-normalized CLIP image embedding for every product using openai/clip-vit-base-patch32.
- Task 1 uses a category-pairing map (e.g. Tshirts → Jeans, Casual Shoes, Watches) combined with cosine similarity within the paired categories to recommend items that are commonly worn together, not just visually similar.
- Task 2 builds a cosine-similarity matrix across all embeddings and greedily collapses any product with similarity ≥ 0.92 to an already-kept item, leaving one representative per near-duplicate cluster.
- Task 3 encodes a free-text query with CLIP's text tower into the same embedding space as the images, then ranks all products by cosine similarity to that text vector.

Task 1 — Smart Product Recommendation Engine

Problem Statement

The base system only finds visually similar products. E-commerce platforms also recommend products that are commonly purchased together (e.g. Running Shoe → Sports Socks, Fitness Watch, Water Bottle). The task was to build a recommender for these complementary products.

How the Recommendations Are Generated

A COMPLEMENTARY_MAPPING dictionary defines which article types pair with which others (for example, Eyewear pairs with Watches, Belts, Perfume and Caps; Tshirts pair with Jeans, Casual Shoes, Watches and Jackets). For a given input product, the system looks up its articleType, filters the catalog down to only the items belonging to the paired categories, and then ranks those candidates by cosine similarity between their CLIP embeddings and the input product's embedding. This ensures recommendations are both categorically complementary (a genuinely different, pairable item type) and visually/stylistically coherent with the input (similar colour, pattern or aesthetic within that category). If no mapping exists for a category, the system falls back to recommending from the full catalog.

Result

Input product: a women's classic sunglasses (Eyewear). The three recommended items are all from the mapped complementary categories and are visually close in style to the input, confirming the logic works as intended.



Figure 1: Input sunglasses (blue) with three complementary recommendations (green).

Task 2 — Unique Product Catalog Creation

Problem Statement

Large marketplaces often contain duplicate or near-duplicate product listings uploaded by different sellers (e.g. three near-identical blue shirts). The task was to build a system that automatically produces a clean catalog containing only unique products.

How Duplicates Are Identified and Removed

A full pairwise cosine-similarity matrix is computed across all 200 CLIP image embeddings. The catalog is then walked in order: each unvisited product is kept as a representative, and every other product whose similarity to it is 0.92 or higher is marked visited and excluded going forward. This effectively clusters near-duplicate listings (same or nearly identical product photographed/listed multiple times) and keeps a single representative per cluster, producing a deduplicated catalog.

Result

Out of the 200 sampled products, the deduplication step reduced the catalog to 151 unique products — removing 49 near-duplicate listings (similarity ≥ 0.92).

Original Catalog Size: 200 | Unique Catalog Size: 151

Search Results for: 'blue casual shirt'

Figure 2: Catalog size before and after deduplication.

Task 3 — Reverse Product Search

Problem Statement

Instead of searching with an image, users should be able to search the catalog using a free-text query, e.g. "blue casual shirt", and get back the most relevant matching products.

How Text-to-Image Search Works

CLIP's text encoder projects the input query into the same embedding space used for the product images. The query embedding is L2-normalized and compared against every pre-computed, normalized image embedding using cosine similarity. The top-k highest scoring products are returned and displayed along with their similarity scores, giving a ranked, text-driven search experience without needing any labelled search index.

Result

Query: "blue casual shirt". The top 3 results returned were all blue/light-toned men's casual shirts, with similarity scores of 0.2780, 0.2725 and 0.2723 respectively — confirming the text query correctly retrieved semantically and visually relevant products.



Figure 3: Top 3 results for the text query "blue casual shirt", with cosine similarity scores.

Submission Checklist

Requirement	Status	Notes
Source Code / Kaggle Notebook (.ipynb)	Included	See notebook link below
Report with explanation of all tasks	Included	This document
Screenshots of results	Included	Figures 1–3
Task 1 — Recommendation logic	Done	Category-pair map + cosine similarity
Task 1 — Visualization of recommendations	Done	Figure 1
Task 2 — Duplicate identification via embeddings	Done	Cosine similarity matrix, threshold 0.92
Task 2 — Unique catalog generated	Done	200 → 151 products
Task 3 — Text-based search interface	Done	CLIP text encoder
Task 3 — Display of matching products	Done	Figure 3

Links

Kaggle Notebook (Source Code): <https://www.kaggle.com/code/manishr00799/notebook4d97fc13a9/>

Dataset: <https://www.kaggle.com/code/sahandakramipour/fashion-product-images-small>

Judging Criteria Alignment

Criteria	Weight	How this submission addresses it
Correctness	40%	All three tasks run end-to-end on the sampled dataset and produce the expected outputs shown
Creativity	10%	Category-pair mapping for complementary recommendations goes beyond plain visual similarity
Technical Implementation	20%	Single CLIP embedding pipeline reused across all three tasks; robust handling of missing/corrupt
Presentation & Explanation	30%	This report explains the approach, logic and results for each task with visual evidence.